UNIT II: Distributed Operating Systems: Topology, Network Types, Communication, RPC, Client server model, Distributed file systems and design strategies.

DISTRIBUTED OPERATING SYSTEM:

The distributed operating system consists of different computers which are connected with a network within a virtual shell. A user interacts with the virtual shell in order to perform different tasks as and when needs arises but the distributed operating system architecture delegates the responsibility of performing a task to one or more computers within the virtual shell.

The components within a distributed operating system can be computers of different operating system which are located at different sites for execution. All the components are connected using a strong local area network as the same plays a vital role in execution of a task within a distributed architecture. The operating system generally provides different functions like process management, input/ output management, memory management, file organization, security & protection and network management. The distributed operating system has different components which are responsible from different functions of an operating system and all these components are connected with a network. Whenever a user submits a command to a distributed operating system, the instruction may require the services of one or more than one components of an operating system. The user gets a feel as if the whole system is a single unit but internally the whole system consists of sub systems which work in tandem in order to achieve the centralized objective of a distributed operating system. Plain distributed processing shares the workload among computers that can communicate with one another. True distributed processing has separate computers to perform different tasks in such a way that their combined work can contribute to a goal. The latter type of processing requires a highly structured environment that allows hardware and software to communicate, share resources, and exchange information freely.

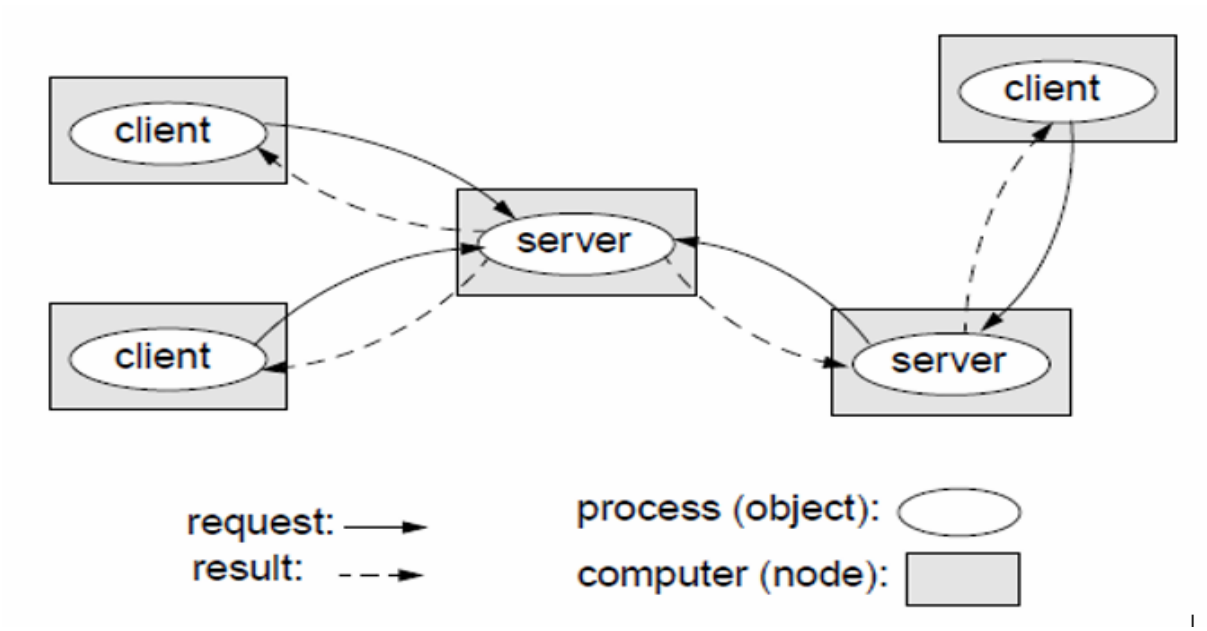
EVOLUTION OF DISTRIBUTED OPERATING SYSTEM:

The computers were having very less computing power in the early days of computing when the computers were introduced. The pace of execution along with processing power was increasing many-fold with the advances in technology. Nowadays, minicomputers were used to process different volumes of data. With the advancement in technology and increase in computer speed along with the processing power of a processor, more complex processing along with huge volumes of data was performed on personal computers. However, data analysis where the volume of data was mammoth was performed on main-frame computers. The main-frame computers were not appropriate options for processing of different task in some case which led researchers in exploring option to find different alternatives and one of the alternatives provided by the research community was distributed operating system architecture. The distributed architecture was about delegating the responsibility among one or more computers connected together within a network. This feature of distributed architecture allowed an end user to get higher speed and better processing power. The architecture helped researchers to increase the processing power to manifold by integrating the resources of different independent computers as a single virtual computer. The distributed architecture is not only about hardware rather it is amalgamation of hardware utilization along with appropriate software which help the system to perform with efficiency and effectiveness. The distributed architecture was further enhanced with the advent of RPC remote procedure calls. The RPCs gained prominence due to the availability of TCP/IP protocol which is known as Transmission Control Protocol/ Internet Protocol. The RPCs helps an individual to execute different commands and instructions on any computer remotely provided the computer is part of a network which a user has permissions to access. The feature has led to more prominence and relevance to the need of

distributed operating system architectures in providing higher processing power to an end-user. Initially the architectural framework was designed to perform batch processing which was later on improved with the inception of minicomputers. The mini computers enabled the researchers to incorporate the concept of master/ slave where a computer was nominated as master and other connected nodes were treated as slaves. The complete execution of a process from inception till maturity was monitored by master and all the slaves will execute the orders of the master as per the requirement of a process or task. As the size of computers was reduced dramatically and with the increase in the processing power of microcomputers the processing within a distributed operating system become more easy, effective and efficient. Nowadays the networks that allow user to connect to internet has reached newer heights in terms of performance, scalability and securitywhich has resulted in leveraging from the advancements in networks by incorporating the high end servers to help in distributed computing with the help of strong, reliable and secure networks.

MODELS OF DISTRIBUTED OPERATING SYSTEM:

The models in distributed operating system can also be termed as architectural models. These models give a brief idea about the connection of different components within a distributing operating system of distributed computing. The information about the architectures helps the researchers in devising policies and procedures to make these systems more scalable, robust, effective and efficient. The different architectural models of distributed operating system are listed below: 1. Interaction Model 2. Failure Model 3. Security Model Interaction model In the interaction model, the issues related to the interaction of process are included like timing of events. An example of an interaction model can be a client-server architecture where a client submits a request to the server and the server responds with a reply.



In a distributed operating system environment, all the clients need to interact with the server in order to process a request. The server in turn manages and monitors the processing within all the components of a distributed operating system connected with a network. However, the communication between the client and the server result in different issues which need to be addressed by the developers appropriately. The interaction model can be categorized into two main categories which are given below: (i) Synchronous Model (ii) Asynchronous Model The Failure Model

Failure Model uses the specifications of different faults that occur during the processing of a process and while communicating between the different components of a process. The failure model handles different situation where failures do occur provided the system is able to identify and detect a failure like Omission failure, Byzantine (Arbitrary) Failure and timing failures etc.

The Security Model:

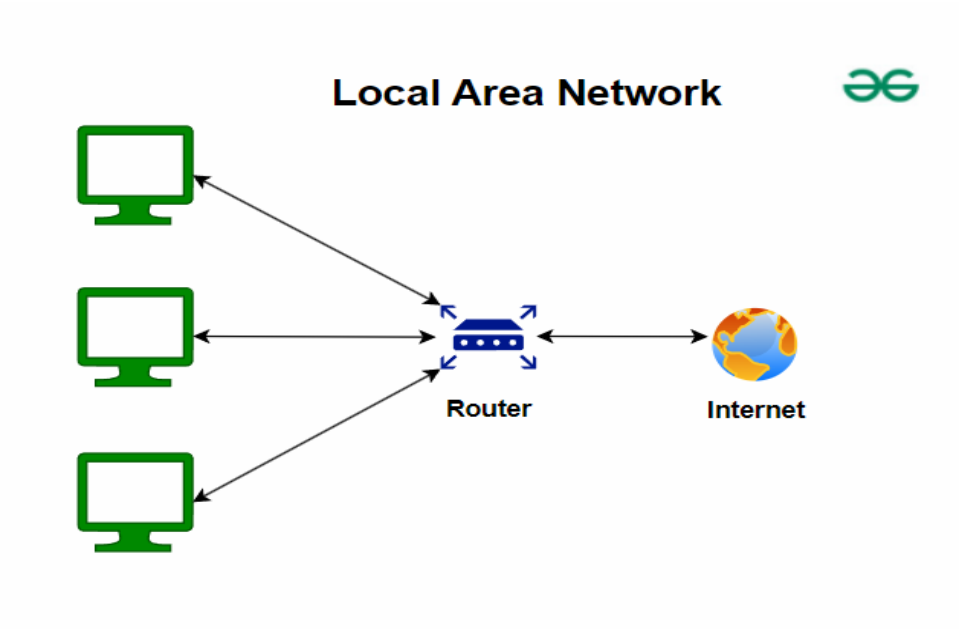
The Security Model uses the specifications of different threats that occur during the processing of a process and while using the communication channels which play a pivotal part in execution of any process in a distributed operating system. Security threats in a distributed operating system are generally attacks that are intended to break the communication channel or to make a communication process unsuccessful which will result in failure of a distributed operating system. Some of the commonly occurred threats are list below:

- Unauthorized connection to network
- Identity Threat
- Denial of Service
- Modification/ Deletion of message in a communication channel
- Trojan Horse
- Virus
- Worm
- Server overloading

In order to make any system secure the recommendation is always to use pro-active approach which is similar to the approach of "Prevention is better than cure" where the impact of threats can be mitigated by properly monitoring the processing of tasks and by taking preventive measuring for mitigating the impact of threats.

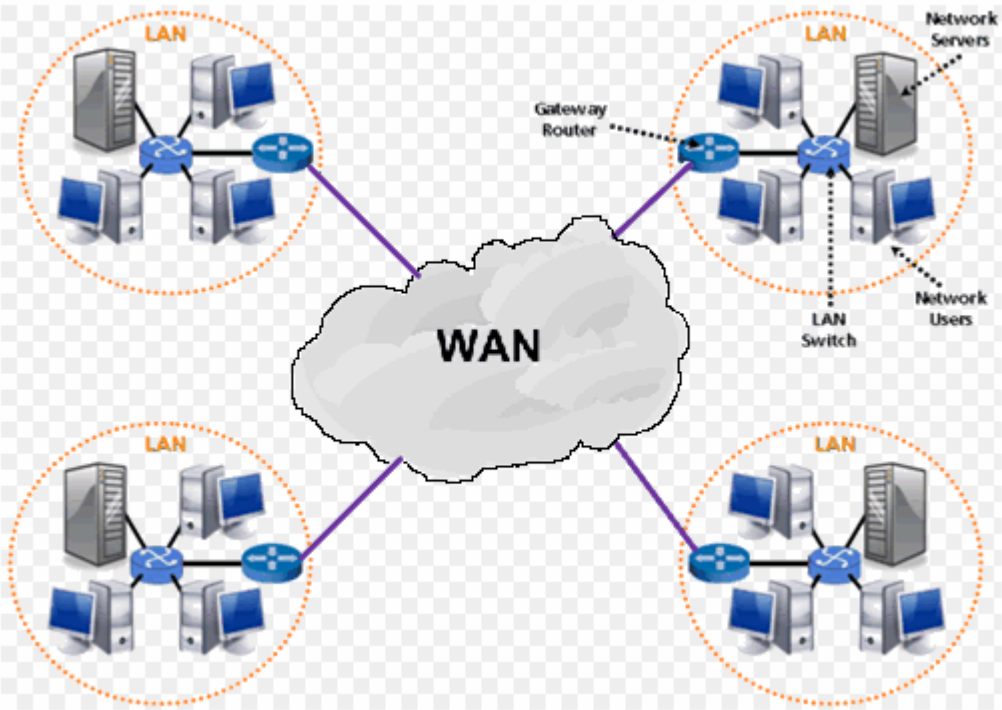
Types of Networks:

A computer network can be as small as several personal computers on a small network or as large as the Internet. Depending on the geographical area they span, computer networks can be classified into two main categories, namely, local area networks and wide area networks. Local Area Networks A Local Area Network (LAN) is the network restricted to a small area such as an office or a factory or a building. It is a privately owned network that is confined to an area of few kilometers. In a LAN, the computers connected have a network operating system installed in them. One computer is designated as the file server which stores all the software that controls the network and the software that can be shared by the computers attached to the network. The other computers connected to the file server are called workstations. The workstations can be less powerful than the file server and may have additional software on their hard drives. On most LANs, cables are used to connect the computers. Generally, a LAN offers a bandwidth of 10 to 10 Gbps. LANs are distinguished from other networks by three main characteristics including their size, topology, and transmission technology.



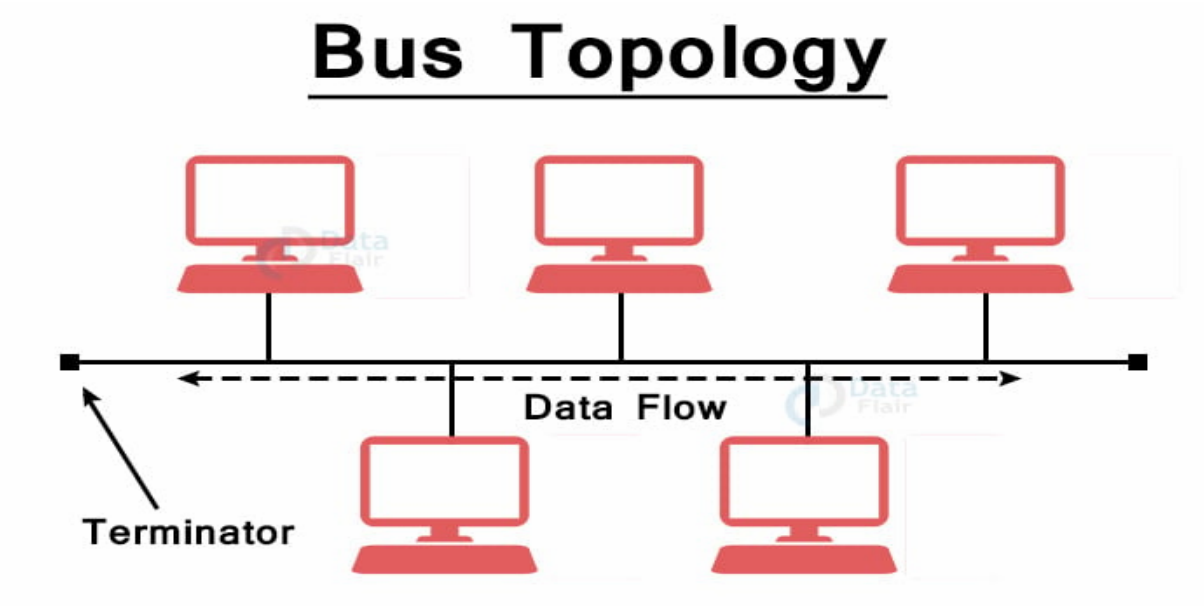
Wide Area Network (WAN):

A Wide Area Network (WAN) spreads over a large geographical area like a country or a continent. It is much bigger than a LAN and interconnects various LANs. This interconnection helps in a faster and more efficient exchange of information at a higher speed and low cost. These networks use telephone lines, satellite transmission and other long-range communication technologies to connect the various networks. For example, a company with offices in New Delhi, Chennai and Mumbai may connect their individual LANs together through a WAN. The largest WAN in existence is the Internet.



Network Topology:

A network topology refers to the way a network is laid out either physically or logically. The selection of a particular topology is important and depends upon the number of factors like cost, reliability and flexibility. The various network topologies include bus, ring, star, tree, mesh, and graph. Bus/Linear Topology The bus topology uses a common single cable to connect all the workstations. Each computer performs its task of sending messages without the help of the central server. Whenever a message is to be transmitted on the network, it is passed back and forth along the cable from one end of the network to the other. However, only one workstation can transmit a message at a particular time in the bus topology. As the message passes through each workstation, the workstations check the message's destination address. If the destination address in the message does not match with the workstation's address, the bus carries the message to the next station until the message reaches its desired workstation. Note that the bus comprises terminators at both ends. The terminator absorbs the message that reaches the end of the medium. This type of topology is popular because many computers can be connected to a single central cable.



Advantages:

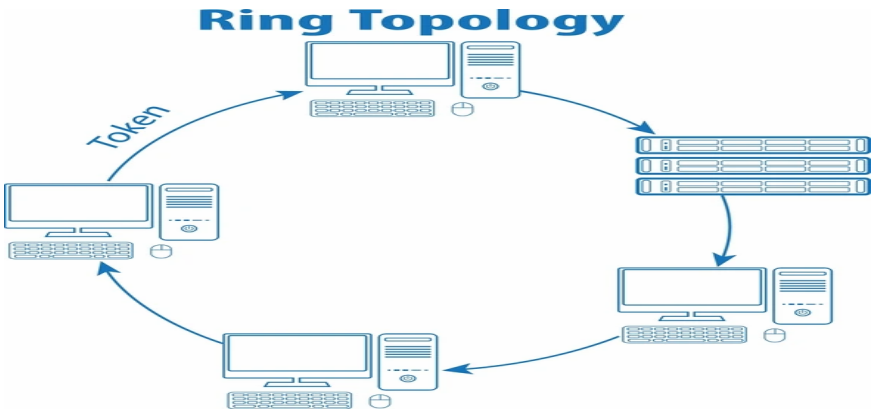
- It is easy to connect and install.
- It involves a low cost of installation.
- It can be easily extended.

Disadvantages :

- The entire network shuts down if there is a failure in the central cable.
- Only a single message can travel at a particular time.
- It is difficult to troubleshoot an error.

Ring/Circular Topology In the ring topology, the computers are connected in the form of a ring without any terminated ends. Every workstation in the ring topology has exactly two neighbours. The data is accepted from one workstation and is transmitted to the destination through a ring in the same direction (clockwise or counter clockwise) until it reaches its destination. Each node in a

ring topology incorporates a repeater. That is, each workstation re-transmits data or message received from a neighbouring workstation, no signal is lost and hence, repeaters are not required. In addition, since the ring topology does not have a terminator that terminates the message received, the source computer needs to remove the message from the network.



Advantages:

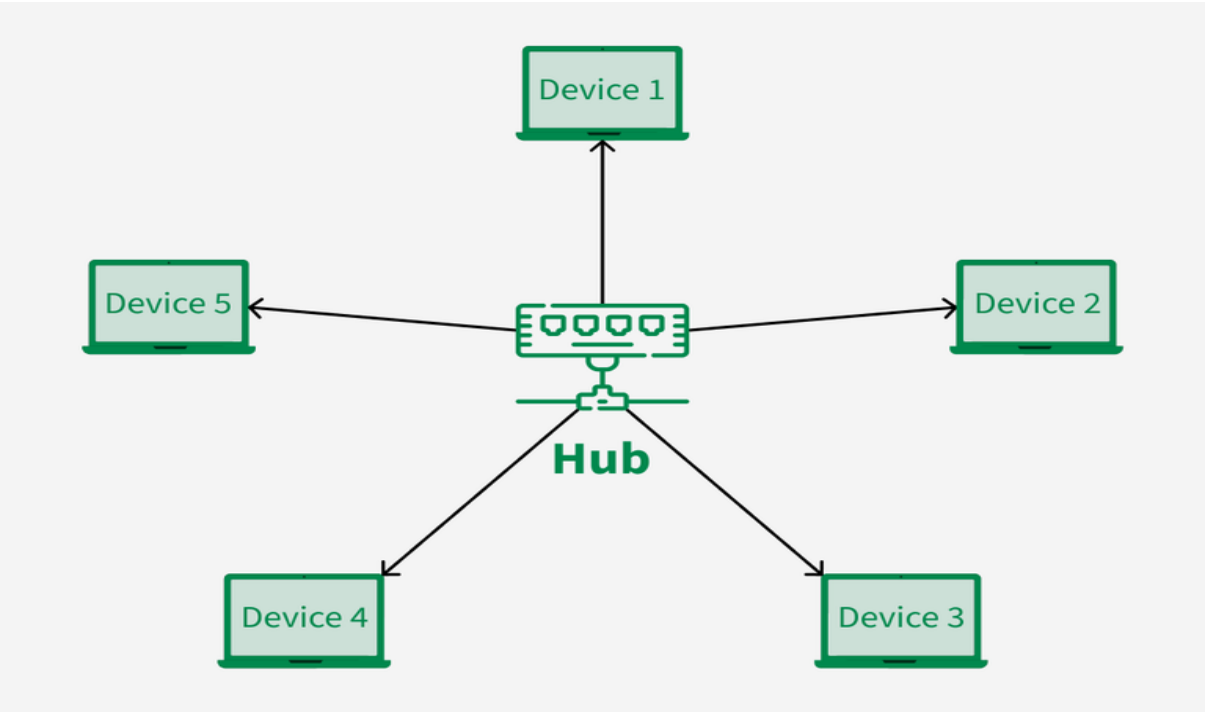
- It is easy to install.
- It uses lesser cable length for the installation.
- Every computer is given equal access to the ring.

Disadvantages:

- The maximum ring length and the number of nodes are limited.
- A failure in any cable or node breaks the loop and can take down the entire network.

Star Topology:

In the star topology, the devices are not directly linked to each other but are connected through a centralized network component known as the hub or the concentrator. Computers connected to the hub by cable segments send their traffic to the hub that resends the message either to all the computers or only to the destination computer. The hub acts as a central controller and if a node wants to send the data to another node, it boosts up the message and sends the message to the intended node. This topology commonly uses twisted pair cable, however, coaxial cable or optical fibre can also be used. It is easy to modify and add new computers to a star network without disturbing the rest of the network. Simply a new line can be added from the computer to the central location and plugged into the hub. However, the number of systems that can be added depends upon the capacity of the hub.



Advantages:

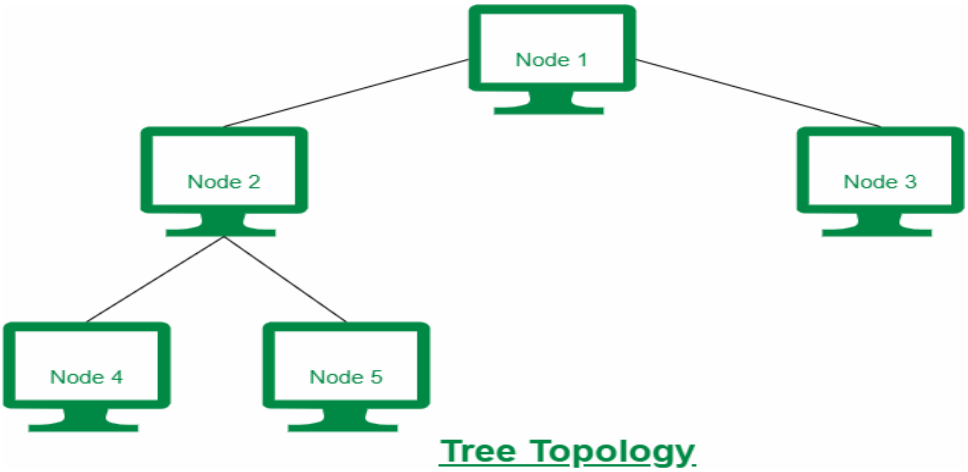
- It is easy to troubleshoot.
- A single node failure does not affect the entire network.
- The fault detection and removal of faulty parts is easier.
- In case a workstation fails, the network is not affected.

Disadvantages:

- It is difficult to expand.
- The cost of the hub and the longer cables makes it expensive over others.
- In case hub fails, the entire network fails.

Tree Topology:

The tree topology combines the characteristics of the bus and star topologies. It consists of groups of star-configured workstations connected to a bus backbone cable. Every node is not directly plugged to the central hub. The majority of nodes connect to a secondary hub which in turn is connected to the central hub. Each secondary hub in this topology functions as the originating point of a branch to which other nodes connect. This topology is commonly used where a hierarchical flow of data takes place.



Advantages:

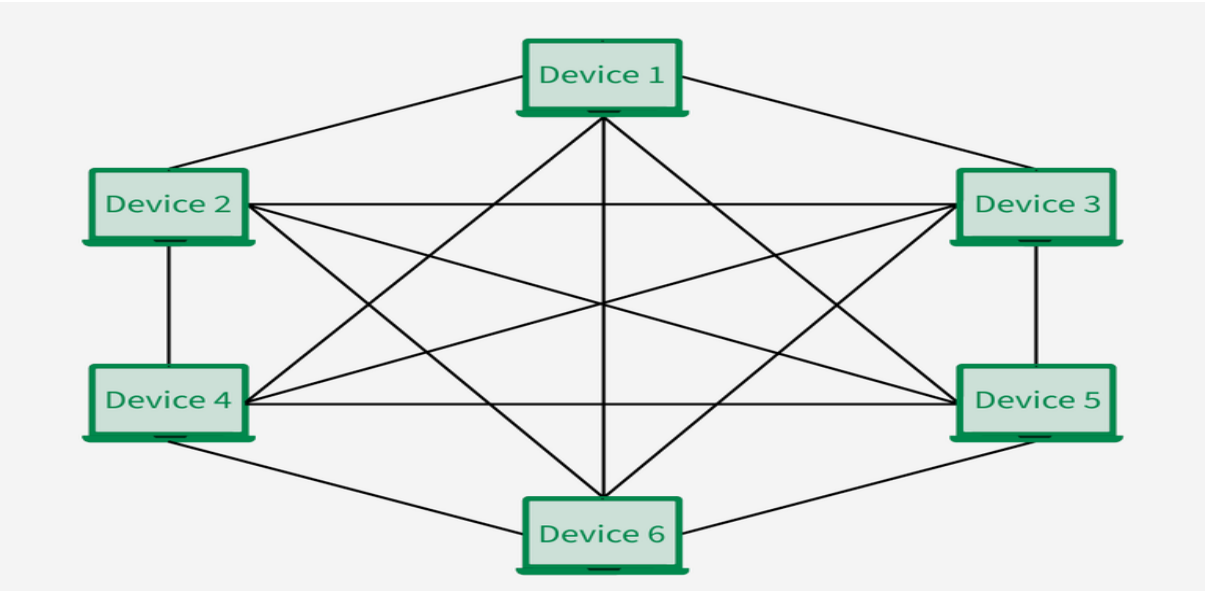
- It eliminates network congestion.
- The network can be easily extended.
- The faulty nodes can easily be isolated from the rest of the network.

Disadvantages:

- It uses large cable length.
- It requires a large amount of hardware components and hence, is expensive.
- The installation and reconfiguration of the network is very difficult.

Mesh Topology :

In the mesh topology, each workstation is linked to every workstation in the network. That is, every node has a dedicated point-to-point link to every other node. The messages sent on a mesh network can take any of the several possible paths from the source to the destination. A fully connected mesh network with n devices has n(n-1)/2 physical links. For example, if an organization implementing the topology has 8 nodes, 8(8-1)/2, that is, 28 links are required. In addition, routers are used to dynamically select the best path to be used for transmitting the data. The mesh topology is commonly used in large Internet-working environment because it provides extensive back up and outing capabilities. This topology is ideal for distributed computers.



Advantages:

- The availability of large number of routes eliminates congestions.
- It is fault tolerant, that is, failure of any route or node does not result in network failure.

Disadvantages:

- It is expensive as more cabling is required.
- It difficult to install.

Graph Topology:

In a graph topology, the nodes are connected randomly in an arbitrary fashion. There can be multiple links and all the links may or may not be connected to all the nodes in the network. However, if all the nodes are linked through one or more links, the layout is known as a connected graph.

Communication Protocols:

A communication protocol (also known as a network protocol) is a set of rules that coordinates the exchange of information. If one computer is sending information to another and they both follow the same protocol, the message gets through; regardless of what types of machines they are and on what operating systems they are running. As long as machines have software that can manage the protocol, communication is possible. The two most popular types of communication protocols are the ISO protocol and TCP/IP protocol.

ISO Protocol:

The International Standards Organization (ISO) provided an Open Systems Interconnection (OSI) reference model for communication between two end users in a network. In 1983, ISO published a document called 'The Basic Reference Model for Open Systems Interconnection' which visualizes network protocols as a seven-layered model. The model lays a framework for the design of network systems that allow for communication across all types of computer systems. It consists of seven separate but related layers, namely, Physical, Data Link, Network, Transport, Session, Presentation and Application. A layer in the OSI model communicates with two other OSI layers, the layer directly

above it and the layer directly below it. For example, the data link layer in System X communicates with the network layer and the physical layer. When a message is sent from one machine to another, it travels down the layers on one machine and then up the layers on the other machine.

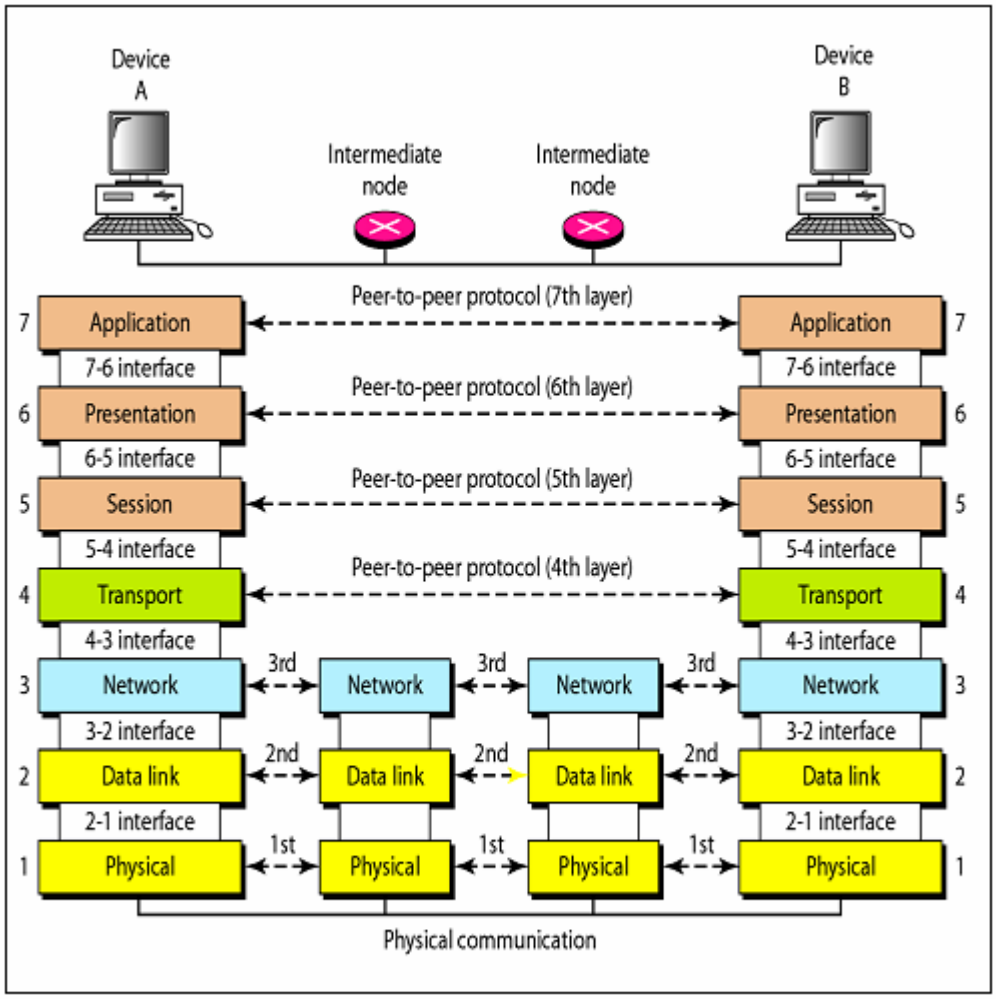


Fig: Communication & Interfaces in the OSI model

As the message travels down the first stack, each layer (except the physical layer) adds header information to it. These headers contain control information that are read and processed by the corresponding layer on the receiving stack. At the receiving stack, the process happens in reverse. As the message travels up the other machine, each layer strips off the header added by its peer layer.

The seven layers of the OSI model are listed here.

• **Physical Layer:** It is the lowest layer of the OSI model that defines the physical characteristics of the network. This layer communicates with data link layer and regulates transmission of stream of bits (0s and 1s) over a physical medium such as cables, optical fibers, etc. In this layer, bits are converted into electromagnetic signal before traveling across physical medium.

• **Data Link Layer:** It takes the streams of bits from the network layer to form frames. These frames are then transmitted sequentially to the receiver. The data link layer at the receiver's end detects and corrects any errors in the transmitted data, which travels from the physical layer.

• **Network Layer:** This layer is responsible for transferring data between the devices that are not locally attached. Network layers manage the network traffic problems such as routing data packets. The network layer device such as router facilitates routing services in a network. The router checks the destination address of the packet received and compares with the routing table (comprises network addresses). The router directs the packet to an appropriate router to reach the destination.
• **Transport Layer:** The transport layer establishes, maintains and terminates communication between the sender and the receiver. This layer manages the end-to-end message delivery in the network

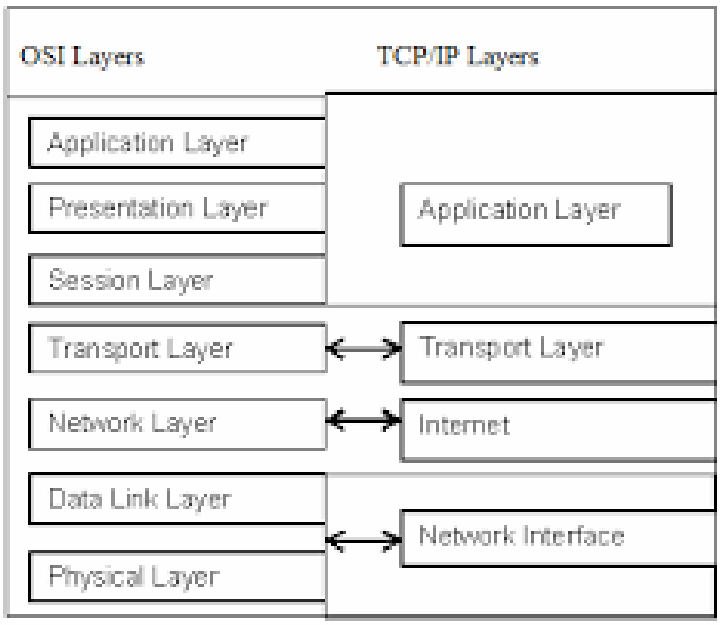
• **Session Layer:** The fifth layer of the OSI model organizes and synchronizes the exchange of data between the sending and the receiving application. This layer keeps each application at one end and confirms know the status of other applications at other end.

• **Presentation Layer:** The sixth layer in the OSI model is responsible for format and code conversion like encoding and decoding data, encrypting and decrypting data, compresses and decompresses data. The layer ensures that information or data sent from the application layer of a computer system is readable by application layer of another computer system. The layer packs and unpacks the data.

• **Application Layer:** This layer is the entrance point that programs use to access the OSI model. It is the "topmost" or the seventh layer of the OSI model. It provides standardized services such as virtual terminal file and job transfer operations.

TCP/IP Protocol:

The Transmission Control Protocol/Internet Protocol (TCP/IP) is the most widely adopted protocol over the Internet. It has fewer layers than that of the ISO protocol, which makes it more efficient. However, it combines several functions in each layer, which makes it more difficult and complex to implement. The various layers in the TCP/IP protocol are listed here. • The Link Layer: It corresponds to the hardware, including the device driver and interface card. The link layer has data packets associated with it depending on the type of network being used, such as Token ring or Ethernet. • The Network Layer: It manages the movement of packets around the network. It is responsible for making sure that packets reach their destinations correctly. • The Transport Layer: It is the mechanism used for two computers to exchange data with regards to software. The two types of protocols that are the transport mechanisms are TCP and UDP. • The Application Layer: It refers to networking protocols that are used to support various services, such as FTP, Telnet, etc.



Distributed File Systems (DFSs) are designed to support the sharing of information across a network in the form of files. If the system is designed well then it provides the same reliability and performance in accessing the files stored at the server as files stored on some local host. In computing, it allows the programs to store and access files remotely from any computer in an intranet. Thus, multiple users on multiple computer systems can share files and resources. The client systems can access the storage over the network using its protocol. The design of the protocol restricts access to the file storage depending on the access lists on both the server side and client side. All the machines have equal access to the storage when the file system is located in shared disk and the access control on these systems resides on the client. Distributed file systems must cover the facilities of scalability, fault tolerance and replication. In distributed file systems, files are stored on one or more systems which are called servers and these files are accessible by various systems which are clients. It is easy to provide backups and security of the data in the distributed file systems as it needs only servers to be secured and backed up. There are a lot of problems in designing a proper distributed file system. Accessing many files over the network may create network bottlenecks and server overload which may slow down the performance of the network. Security of the data is also an important issue. Network and server failures may also create problems in designing a distributed file system. Most often, client computers are more reliable in comparison to the network connecting them. Some services, such as authentication service, printing service and name service can easily be implemented in DFS or Distributed File System. In the organizations where Web servers are used for external and internal access through intranet, these Web servers access and store data from a distributed file system. In distributed object oriented programming, there is a requirement for the storage and distribution of the shared objects. This can be achieved by serializing the objects and storing and retrieving the serialized objects using the files. But if the objects change rapidly then this will not work and therefore, some more direct approaches have been developed. Java RMI and CORBA provide access to remote, shared objects but they do not ensure the persistence of the objects. The current developments include Distributed Shared Memory, i.e., DSM and persistent objects stores for the distribution of the distributed information. DSM copies segments or memory pages at each host which provides an emulation of the shared memory. For optimizing the performance of the distributed programs all storage systems depend on caching. When the replicas of the memory pages or segments are used then it is difficult to achieve

strict consistency. The Web uses caches both at the client and the proxy server sides. The explicit user actions maintain consistency between the replicas stored at original server, proxy server and the client caches. A client must keep the local copies of the data up to date explicitly as the updation on the server side is not notified to the client side.

FILE MODES:

The basic file model is based on the structure used in a file system within a distributed file sharing environment and the same are categorized as structured and unstructured files. The second categorization of file models within a distributed file sharing environment is based on modifiability are mutable and immutable files.

(a) **Structured File System:** In this file system, the details of a file are known to the storage server where the files are stored. Every file in the system is a collection of records which are in an ordered sequence. The record is the lower most unit is this file system which can vary in size from one file to the other. The sharing of files in this file system are not as simple as unstructured file system. The structured file system is used using two methods as indexed and non-indexed method. The indexed file system stores the records in an indexed sequence where any record can be accessed by specifying the value of one or more key field and the same can be addressed by giving the values of the key fields. In structured file system method the records of a file are maintained using structures and one of the commonly used architecture is B-Tree. However, in non-indexed file system a record is traversed by mentioning the position of a record in a file. For example if the 8th record in a file needs to be accessed the traversing is executed by exploring the first record of the file first and then moving to eighth record in order to retrieve the data from the storage server.

(b**) Unstructured File System:** This file system is the simplest form of a file system where the details about a sub structure are not known to the storing server where the files are stored. The distributed operating system kernel does not require to know about the sub structures of or details of file and data stored on the storage servers which gives applications the access to understand the details of the sub structures of the data stored.

Some of the Introduction to DFS examples where this method of storage was used are MS-DOS and UNIX. Another categorization of file models based on modifiability are given below:

(a) **Mutable Files:** Once a file is stored on a storage server, the possibility of modifying the contents of a file is required. When the content of a file are modified the file is not re-created rather the existing file is updated by overwriting the existing file with a new file. This method is known as mutable file method. This method is commonly used in most of the operating system at present as the method reduces the overheads required to manage the number of in case every modification request re-creates a file again and again.

(b) **Immutable Files:** In this file system the contents are modified by re-creating a file. In this method every modification requests will create a new file and the information about the previous versions of a file is stored as history of the file modified. A separate subroutine is used for managing the versions of a file which helps the operating system to find out the recent updated file. This increases the load on the operating system and increase the quantum of storage space required to store a file.

Design Principles of Distributed File System

1. Scalability:

The system must handle increasing amounts of data and users efficiently without degradation in performance.

Example:

Hadoop Distributed File System (HDFS): HDFS is designed to scale out by adding more DataNodes to the cluster. Each DataNode stores data blocks, and the system can handle petabytes of data across thousands of nodes. When more storage or processing power is needed, new nodes can be added to the cluster without significant downtime or performance degradation.

2. Consistency:

Ensuring that all users see the same data at the same time. This can be achieved through different consistency models.

Example:

Google File System (GFS): GFS provides a relaxed consistency model to achieve high availability and performance. It allows concurrent mutations and uses version numbers and timestamps to maintain consistency. Changes are made in a primary replica and then propagated to secondary replicas, ensuring eventual consistency.

3. Availability

Ensuring that the system is operational and accessible even during failures.

Example:

Amazon S3: Amazon S3 achieves high availability by replicating data across multiple Availability Zones (AZs). If one AZ fails, data is still accessible from another, ensuring minimal downtime and high availability.

4. Performance

Optimizing the system for speed and efficiency in data access.

Example:

Ceph: Ceph is designed to provide high performance by using techniques such as object storage, which allows for efficient, parallel data access. It uses a dynamic distributed hashing algorithm called CRUSH (Controlled Replication Under Scalable Hashing) to distribute data evenly across storage nodes, optimizing data retrieval times.

5. Security

Protecting data from unauthorized access and ensuring data integrity.

Example:

Azure Blob Storage: Azure Blob Storage offers comprehensive security features, including role-based access control (RBAC), encryption of data at rest and in transit, and integration with Azure Active Directory for authentication. This ensures that only authorized users can access or modify the data.

6. Data Management

Efficiently distributing, replicating, and caching data to ensure optimal performance and reliability.

Example:

Cassandra: Apache Cassandra is a distributed NoSQL database that uses consistent hashing to distribute data evenly across all nodes in the cluster. It also provides tunable consistency levels and replication strategies to ensure data is available and performant even during node failures.

7. Metadata Management

Efficient management of metadata, which is crucial for tracking the location, size, and permissions of files.

Example:

HDFS: HDFS uses a centralized NameNode to manage metadata. The NameNode stores information about the file system namespace and the locations of data blocks. While this centralization simplifies management, it also requires robust fault tolerance mechanisms to ensure the NameNode is always available.

8. File Access and Operations

Providing efficient and secure methods for file operations, including reading, writing, and modifying files.

Example:

Google File System (GFS): GFS supports a single-writer, multiple-reader model where files are divided into chunks. Clients can access these chunks directly from the chunk servers after obtaining the metadata from the master server. This approach allows for efficient data access and modification.

9. Fault Tolerance and Recovery

Ensuring the system can detect, handle, and recover from failures without data loss or significant downtime.

Example:

HDFS: HDFS is designed for fault tolerance with data replication. Each data block is replicated across multiple DataNodes. If a DataNode fails, the system automatically re-replicates the blocks from the remaining replicas to ensure data integrity and availability.